EXPERIMENT 6

PROBLEM 1: IRIS FLOWER CLASSIFICATION

1. Title
Write a Python program to implement the Naïve Bayes algorithm for iris flower classification and display the confusion matrix and accuracy.

2. Aim
To classify Iris flowers into Setosa, Versicolor, or Virginica using sepal and petal measurements.

3. Algorithm
1. Start the program.
2. Import the required libraries and load or prepare the dataset.
3. Separate the input features (X) and target labels (y).
4. Split the dataset into training and testing sets.
5. Create a Gaussian Naïve Bayes classifier.
6. Train the classifier using the training data.
7. Predict the class labels for the testing data.
8. Calculate and display the confusion matrix and accuracy.
9. Stop the program.

4. Python Code
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

data = load_iris()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Iris Flower Classification")
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
```

5. Input

Dataset: Iris flower dataset from scikit-learn.

Item	Details
Input features	Sepal length, sepal width, petal length, petal width
Target classes	Setosa, Versicolor, Virginica
Training samples	120
Testing samples	30
Classifier	Gaussian Naïve Bayes

6. Output

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]
```

Accuracy: 96.67 %

7. Result

Thus, the Naïve Bayes algorithm was implemented successfully in Python for iris flower classification. The confusion matrix and accuracy were displayed. The model achieved 96.67% accuracy on the selected test split.

PROBLEM 2: STUDENT PASS/FAIL PREDICTION

1. Title
Write a Python program to implement the Naïve Bayes algorithm for student pass/fail prediction and display the confusion matrix and accuracy.

2. Aim
To predict whether a student passes or fails based on study hours and attendance percentage.

3. Algorithm
10. Start the program.
11. Import the required libraries and load or prepare the dataset.
12. Separate the input features (X) and target labels (y).
13. Split the dataset into training and testing sets.
14. Create a Gaussian Naïve Bayes classifier.
15. Train the classifier using the training data.
16. Predict the class labels for the testing data.
17. Calculate and display the confusion matrix and accuracy.
18. Stop the program.

4. Python Code
```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

# Features: study hours, attendance percentage
X = np.array([[1,40], [2,45], [2,50], [3,55], [3,60], [4,65],
              [5,70], [5,75], [6,80], [7,90], [1,35], [6,85]])
# 0 = Fail, 1 = Pass
y = np.array([0,0,0,0,0,0,1,1,1,1,0,1])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Student Pass/Fail Prediction")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
```

5. Input

Dataset: Small sample student dataset created in the program.

Item	Details
Input features	Study hours and attendance percentage
Target classes	Fail (0), Pass (1)
Training samples	9
Testing samples	3
Classifier	Gaussian Naïve Bayes

6. Output

```
Confusion Matrix:
 [[2 0]
  [0 1]]
Accuracy: 100.0 %
```

7. Result

Thus, the Naïve Bayes algorithm was implemented successfully in Python for student pass/fail prediction. The confusion matrix and accuracy were displayed. The model achieved 100.0% accuracy on the selected test split.

PROBLEM 3: BREAST CANCER CLASSIFICATION

1. Title
Write a Python program to implement the Naïve Bayes algorithm for breast cancer classification and display the confusion matrix and accuracy.

2. Aim
To classify breast cancer samples as malignant or benign using numerical cell measurements.

3. Algorithm
19. Start the program.
20. Import the required libraries and load or prepare the dataset.
21. Separate the input features (X) and target labels (y).
22. Split the dataset into training and testing sets.
23. Create a Gaussian Naïve Bayes classifier.
24. Train the classifier using the training data.
25. Predict the class labels for the testing data.
26. Calculate and display the confusion matrix and accuracy.
27. Stop the program.

4. Python Code
```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Breast Cancer Classification")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
```

5. Input

Dataset: Breast cancer Wisconsin dataset from scikit-learn.

Item	Details
Input features	30 numerical cell-nucleus measurements
Target classes	Malignant, Benign
Training samples	455
Testing samples	114
Classifier	Gaussian Naïve Bayes

6. Output

Confusion Matrix:

```
[[38  4]
 [ 3 69]]
```

Accuracy: 93.86 %

7. Result

Thus, the Naïve Bayes algorithm was implemented successfully in Python for breast cancer classification. The confusion matrix and accuracy were displayed. The model achieved 93.86% accuracy on the selected test split.

PROBLEM 4: WINE CLASSIFICATION

1. Title
Write a Python program to implement the Naïve Bayes algorithm for wine classification and display the confusion matrix and accuracy.

2. Aim
To classify wine samples into one of three classes using chemical measurements.

3. Algorithm
28. Start the program.
29. Import the required libraries and load or prepare the dataset.
30. Separate the input features (X) and target labels (y).
31. Split the dataset into training and testing sets.
32. Create a Gaussian Naïve Bayes classifier.
33. Train the classifier using the training data.
34. Predict the class labels for the testing data.
35. Calculate and display the confusion matrix and accuracy.
36. Stop the program.

4. Python Code
```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

data = load_wine()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Wine Classification")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
```

5. Input

Dataset: Wine recognition dataset from scikit-learn.

Item	Details
Input features	13 chemical measurements
Target classes	Class 0, Class 1, Class 2
Training samples	142
Testing samples	36
Classifier	Gaussian Naïve Bayes

6. Output

Confusion Matrix:

```
[[12  0  0]
 [ 1 13  0]
 [ 0  0 10]]
```

Accuracy: 97.22 %

7. Result

Thus, the Naïve Bayes algorithm was implemented successfully in Python for wine classification. The confusion matrix and accuracy were displayed. The model achieved 97.22% accuracy on the selected test split.

PROBLEM 5: HANDWRITTEN DIGIT CLASSIFICATION

1. Title
Write a Python program to implement the Naïve Bayes algorithm for handwritten digit classification and display the confusion matrix and accuracy.

2. Aim
To classify handwritten digit images into labels from 0 to 9 using pixel values.

3. Algorithm
37. Start the program.
38. Import the required libraries and load or prepare the dataset.
39. Separate the input features (X) and target labels (y).
40. Split the dataset into training and testing sets.
41. Create a Gaussian Naïve Bayes classifier.
42. Train the classifier using the training data.
43. Predict the class labels for the testing data.
44. Calculate and display the confusion matrix and accuracy.
45. Stop the program.

4. Python Code
```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

data = load_digits()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Handwritten Digit Classification")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
```

5. Input

Dataset: Digits dataset from scikit-learn (8 × 8 pixel images).

Item	Details
Input features	64 pixel-intensity values
Target classes	Digits 0 through 9
Training samples	1437
Testing samples	360
Classifier	Gaussian Naïve Bayes

6. Output

Confusion Matrix:

```
[[33  0  0  0  1  1  0  0  1  0]
 [ 0 29  1  0  0  0  1  0  3  2]
 [ 0  3 19  0  0  0  0  0 13  0]
 [ 0  0  1 26  0  0  0  1  8  1]
 [ 0  2  0  0 26  0  1  6  1  0]
 [ 0  0  0  0  0 35  0  1  0  1]
 [ 0  1  0  0  0  0 35  0  0  0]
 [ 0  0  0  0  0  1  0 35  0  0]
 [ 0  4  1  0  0  0  0  1 29  0]
 [ 0  3  1  0  0  0  0  4  3 25]]
```

Accuracy: 81.11 %

7. Result

Thus, the Naïve Bayes algorithm was implemented successfully in Python for handwritten digit classification. The confusion matrix and accuracy were displayed. The model achieved 81.11% accuracy on the selected test split.